The Pennsylvania State University

The Graduate School

College of Engineering

The Harold and Inge Marcus Department of Industrial and Manufacturing Engineering

Evaluating Supervised Machine Learning Methods to Predict Ethnicity from Last Names

A MS Research Paper in

Industrial Engineering

By

Ana Gabriela Camargo Sandoval

© 2023 Ana G. Camargo Sandoval

Submitted in Partial Fulfillment
of the Requirements
for the Degree of

Master of Science

October 2023

Evaluating Supervised Machine Learning Methods to Predict Ethnicity from Last Names

Ana G. Camargo S. [a]
[a] Department of Industrial and Manufacturing Engineering
The Pennsylvania State University
State College, PA, USA

October 2023

Abstract

We investigate the performance of several different machine learning (ML) algorithms for classifying a person's ethnicity solely based on their last name, in order to select the most reliable classifier. As the input data is categorical (strings of text), a pre-processing was first done to the input data using indicator variables (or "one-hot encoding", as known in machine learning), to transform it into numerical data. The following classification methods were used to train a model with a subset of the data: logistic regression, naive Bayes, k-NN and grouped Lasso penalized logistic (GLPL). Our results show that as k-NN is a non-parametric method, it does not perform well on unbalanced data as the training data used are. A logistic regression classifier and the GLPL classifier performed best in terms of accuracy, true positive rate, and false positive rate. These findings encourage to consider and utilize GLPL when performing classification with string-text data, from which there is also an interest that the model learns from the entire sequence of characters.

Keywords: machine learning, supervised learning, classification, ethnicity, indicator variables, confusion matrix .

1 Introduction

The attributes of a person's identity are not always revealed by the users of an information system. Gender, ethnicity and other attributes need to be inferred, in a variety of applications, solely from the name of the person. In this paper, we focus on the problem of determining the ethnicity of a person by considering only her or his last name, and apply different machine learning techniques to solve this problem.

Ethnicity has been researched among different interests and applications. Bertolero (2020) studied ethnicity in terms of discrimination and bias in the academy, showing that belonging to a specific minority or ethnic group affects their recruitment and promotion opportunities, publication rates, grant funding, and awards. Cabreros et al. (2022) stated that as health care algorithms have been increasingly recognized for unintentional discrimination, actions have been taken to eliminate ethnic and race as inputs. However, they argued that this practice could lead to an increase in algorithmic bias and demonstrated that imputing race and ethnicity data actually improved the algorithm.

Nowadays, a huge amount of textual documents are being generated. However, there remains a need to structure them appropriately to enable effective classification into categories; this is where text classification gets relevance. Shah et al. (2020) states that this text classification process is broadly divided into four main parts: pre-treatment, text representation, implementation of the classifier and classification. According to Kadir et al. (2020), machine learning (ML) is increasingly being used for different tasks, with classification being one of the most fundamental applications in many fields. Hu et al. (2021) inferred the gender from information that did not include this input by predicting using solely the provided names.

This study presents several ML classification methods to predict ethnicity given exclusively a particular last name. The goal is to compare different ML techniques applicable to textual sequences and evaluate which one provides better predictions (classifications) for Hispanic and non-Hispanic last names.

The paper is structured as follows. Section 2 presents the general methodology on how to pre-process the categorical data, and explains the classification methods to be used for making predictions which are later evaluated in terms of their performance. Section 3 provides details on the case study in which was applied the different ML methods. This

section includes data sources, description of data pre-processing, as well as an explanation of how performance is measured via the "confusion matrix" and how each ML classification algorithm was implemented. Section 4 follows, showing the results obtained and discussing any findings, insights, and comparisons. Finally, section 5 summarizes the work done and outlines further research. R code that implements all ML algorithms used in this research is provided in the Appendix.

2 Methodology

In this supervised learning problem we were dealing with a qualitative regressor instead of a numerical one; that is, the last names strings are the regressors. Simply coding the letters with numbers (i.e., their ASCII code) will not recognize their sequence and hence, linear regression is not a good way to classify the data. On the other hand, the response (or "label") associated with each input is simply a number indicating the ethnic class each last name corresponds to. For the different machine learning methods explored below, we require two basic components:

- X matrix: input or regressors. This consists of p predictors that could be either numerical or categorical for the total set of n observations. Therefore, this is a $n \times p$ matrix, as the one below.

$$X = \begin{bmatrix} x_{11} & x_{12} & \dots & x_{1p} \\ x_{21} & x_{22} & \dots & x_{2p} \\ \vdots & \vdots & \ddots & \vdots \\ x_{n1} & x_{n2} & \dots & x_{np} \end{bmatrix}$$

- Y vector: outcome or response, the "classes". This vector takes categorical values, which could be encoded for ease of coding manipulation as shown below.

$$Y = \begin{cases} 0 & \text{, if "class 0"} \\ 1 & \text{, if "class 1"} \\ \vdots & \vdots \\ k & \text{, if "class } k\text{"} \end{cases}$$

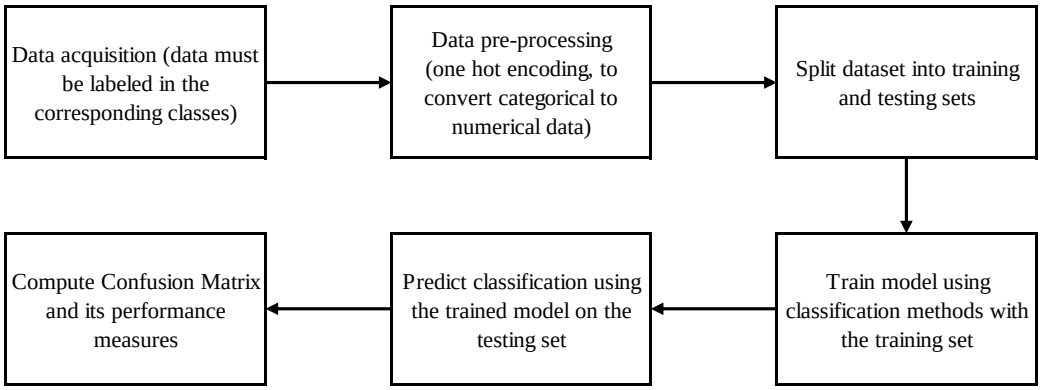

Figure 1: Methodology flowchart

.

Given that input data is categorical (i.e., last names), a data pre-processing was performed to code each last name into a series of 0/1 indicator variables, which are numerical and can be processed by the different methods. Figure 1 shows a flowchart of the methodology for this classification problem, which will be explained more in depth throughout this section.

2.1 *Data acquisition and pre-processing*

To train each ML model, we needed a list of last names labeled in the classes we are interested to classify. ML classification methods are developed so that they use numerical data as input. However, we are dealing with string data corresponding to the list of last names. One can simply create an indicator or "dummy" variable to represent a qualitative predictor as a binary variable (such as 0/1, for instance) to indicate the presence or absence of an attribute. When having more than two levels or categories, we can create same amount of dummy variables to represent each of them (James et al. (2021)). One widely used method to transform categorical data into numerical is the *one-hot encoding*, as this is a simple technique to represent strings that have a finite set of values (Bagui et al. (2021)).

2.1.1 *One-hot encoding*

One-hot encoding is a widely used ML coding technique to convert categorical variables into binary vectors. Combinations of values of these new binary columns indicates each

different class. The algorithm works by first mapping each categorical variable to a specific integer. Then each integer value is represented as a binary vector, meaning that the index of the matched integer is represented as '1' and everything else as '0' (Singh et al. (2020)). Table 1 shows a simple example of one-hot encoding for categorical data (DanB. (2017)).

Color	x_1	x_2	x_3
Red	1	0	0
Red	1	0	0
Yellow	0	1	0
Green	0	0	1
Yellow	0	1	0

Table 1: One-hot encoding example (x_1, x_2, and x_3 are 0/1 "indicator variables").

.

2.2 *Classification methods*

Classification methods in ML are used when the goal is to take an input X and assign it to one of K discrete classes C_k, where $k = 1, ..., K$ (Bishop (2006)). The input is known as the predictors or regressors, while the class is the categorical output or label. In supervised ML, we have some training data that contains the data for both input (X) and output (Y). One could train a model using this data to perform classification methods, later applying the fitted model on a new and previously unseen set of test data. Our goal is to predict the output for which only X is known (Lindholm et al. (2022)).

When constructing a model with a specific dataset, we could lead to reduce the error as we can test the model on the same data points and then compare these predictions with the actual ones. However, it is more likely that in real life problems we would need to face the fact that the model needs to perform well on new data which was not the one used to train the model. When only using one source of data for both, we could lead the model to overfitting. But we also do not want to have an underfitting model. There is a trade-off in the general error incurred with the model, divided between the training and the testing errors (Abu-Mostafa et al. (2012)). Therefore, it is quite important to select an accurate method to split the dataset into a training and a testing set.

A common rule of thumb on how to split a dataset is that the training dataset should be bigger, using 2/3 of the data and leaving the remaining 1/3 for testing the model.
We next explain each of the ML supervised classification methods that were used for this classification problem.

2.2.1 *Logistic Regression*

Logistic regression models the probability that Y belongs to a particular class given a set of observations, instead of modeling the response Y itself. This probability $P(X)$ will take values between 0 and 1, and is modeled using the logistic function in expression 2.1:

$$P(X) = \frac{e^{\beta_0+\beta_1 X_1+...+\beta_p X_p}}{1+e^{\beta_0+\beta_1 X_1+...+\beta_p X_p}} \tag{2.1}$$

Where $X_1, ...X_p$ are the p predictors/regressors, and $\beta_0, ..., \beta_p$ are the regression coefficients, estimated using a maximum likelihood method (James et al. (2021)).
The logistic regression classifier has achieved importance in recent times, and the use has increased extensively. A study by Shah et al. (2020) compared different ML algorithms for text classification; results showed that the logistic regression classifier had the highest accuracy (97%) for the analyzed data set.

2.2.2 *Naive Bayes*

The naive Bayes classifier takes its name from the Bayes' theorem which provides an expression for the posterior probability of some training observations belonging to the kth class. Each one of the K classes will take a particular density function, $f_k(x)$, that is estimated making a simple but powerful assumption: within the kth class, the p predictors are independent (James et al. (2021)). Equation 2.2 shows an expression for the posterior probability for this naive Bayes classifier:

$$P(Y = k/X = x) = \frac{\pi_k \times f_{k1}(x_1) \times ... \times f_{kp}(x_p)}{\sum_{l=1}^{K} \pi_l \times f_{l1}(x_1) \times ... \times f_{lp}(x_p)} \tag{2.2}$$

where $k = 1, ..., K$ are the classes, π_k represent the prior probability that a randomly chosen observation comes from the kth class., and $f_{k_1}, ..., f_{k_p}$ are the density functions of the p predictors among observations in the kth class.
The naive Bayes classifier is a robust, easy to implement, fast and accurate classification

method due to the class independence assumption that simplifies algorithm computations (Bagui et al. (2021)). Hence, it has been widely used in many fields for text classification.

2.2.3 *k-NN*

The k-NN method, a nonparametric method, is an abbreviation for k-nearest neighbors; being k an hyperparameter that could take values from 1 to N (the total number of possible neighbors in the training data). This method starts calculating the euclidean distance between each testing input and all training inputs. Then, the k-NN classifier identifies the k points in the training data that are closest to x_0 (the testing observation), represented by N_0. The estimated conditional probability for class j as the fraction of points in N_0 whose response values equal j is given by expression 2.3 (James et al. (2021)). Finally it classifies the test input x_0 into the majority vote class.

$$P(Y = j/X = x_0) = \frac{1}{K} \sum_{i \in N_0} I(y_i = j) \tag{2.3}$$

Selecting an appropriate value of k is a crucial task for the model, as it has a big impact on making predictions. The variation among predictions for different values of k is due to the existence of certain amount of randomness in the data (Lindholm et al. (2022)). Finding an appropriate value for k could be done by different methods, but the goal is to choose a value of k with the best classification performance. We use the training data to classify the input from the testing data, and then compute error rates for various choices of k (Shmueli et al. (2018)). The error rates can be plotted to get an ROC curve, which helps us to identify the value of k with the best classification performance. An ideal ROC curve will hug the top left corner, close to the highest true positive rate and lowest false positive rate (James et al. (2021)).

The k-NN as a non-parametric method explicitly uses the training data when making predictions. It does not fit a model as in parametric methods, where it fits a model having a fixed number of parameters and then making predictions using this function. Therefore, the training data used for making the predictions is then disregarded and no longer used (Lindholm et al. (2022)).

2.2.4 *Grouped Lasso Penalized Logistic*

The grouped Lasso penalized logistic (GLPL in short) is a special case of penalized regression where the penalty function is an intermediate between the l_1 norm penalty that is used in the Lasso and the l_2 norm penalty that is used in Ridge regression (Yuan and Lin (2006)). The group Lasso estimate is defined as the solution to minimizing this objective function:

$$\frac{1}{2}\left\|Y - \sum_{j=1}^{J} X_j\beta_j\right\|^2 + \lambda \sum_{j=1}^{J} \|\beta_j\|_{K_j} \tag{2.4}$$

From equation 2.4, Y is the vector of classes, J is the grouping vector, X_j is the input matrix of regressors for the jth group, λ is a tuning parameter of the penalty term (with $\lambda \geq 0$), and β_j is the vector of regression coefficients of the jth group (Breheny (2023)).

The parameter λ could be tunned using different techniques. This could be set from a starting value of λ close to 0 and then be varied up to a maximum defined value, so an optimal λ can be found that performs best on training the model. One of these tuning methods is done by cross-validation. Cross-validation is a way to tune the hyperparameters using only the training data. There are different variations of cross-validation, but the most common one is 10-Fold cross-validation (Kangralkar (2021)). Based on the interest of preserving together the string of characters for each entry, GLPL provides a convenient methodology to group the regressors all together when fitting a classification model.

2.3 *Performance evaluation*

With any classification method we can either classify well or perform misclassification. When dealing with two classes, for instance, misclassification may occur in one of these two scenarios. An observation could be classified as "class +1" when it actually belongs to "class -1"; or the opposite could happen, classifying as a "class -1" a "class +1" observation. Determining which of these two error types is occurring is often of interest, as we want to evaluate the performance of the model on new data. One way to measure this performance is using a confusion matrix.

2.3.1 *Confusion matrix*

A confusion matrix provides a convenient method for presenting the true class against the predicted class, hence providing the proportion of correctly and incorrectly classified observations. For a two class confusion matrix, we have four components in the matrix (James et al. (2021)), shown in Table 2: true positive (TP), false negative (FN), false positive (FP), and true negative (TN).

		True class		
		Positive	Negative	Total
Predicted class	Positive	TP	FP	$P* = TP + FP$
	Negative	FN	TN	$N* = FN + TN$
	Total	$P = TP + FN$	$N = FP + TN$	$P + N = P* + N*$

Table 2: Confusion Matrix with two classes.

From these four components, we can compute some important measures that are frequently used as performance metrics of any classification algorithm:

- Accuracy $= (TP + TN)/Total$
- True positive rate: $TPR = TP/P$
- False positive rate: $FPR = FP/N$

A classifier is considered to perform well if having high values on the accuracy and TPR, while having a small FPR. Only evaluating the classifier in terms of accuracy is not enough as this performance measure could be biased when having an unbalanced dataset.

3 Case Study: Hispanic and non-Hispanic last names

3.1 *Data source and description*

A list of Hispanic last names was downloaded from the FCDS website in PDF format (Miller School of Medicine (2023)). The document was transformed to CSV format for data handling and analysis purposes. This summarizes a total of 12,497 last names then labeled as "Hispanic". A long list of last names (in general, not only Hispanic) was retrieved from a

public GitHub repository by ARIN Engineering (Newton (2011)). These two datasets where then appended and merged so that all last names not included in the Hispanic dataset were labeled as "non-Hispanic". This summarizes a total of 84,596 non-Hispanic last names.
Total dataset consists of 97,093 last names labeled as:

- Hispanic: 12,497 (12.9%)
- Non-Hispanic: 84,596 (87.1%)

As we can see from above proportions, this dataset is unbalanced within the two classes. This means that one class is much smaller than the other, therefore the Hispanic class is the minority class. There are different ways to handle with unbalanced data. A study on ethnicity packages done by Xie (2022) dealt with imbalanced data by doing undersampling on the majority class to generate a data subset that is balanced. An additional benefit with this undersampling approach is the reduction of computational time, given the huge size of our entire dataset.
For the undersampling of the majority class, the following approach was taken. There was sampled 50% of the Hispanic class and then sampled this same number from the non-Hispanic class. The sample selection was done using Rstudio software generating a random sample without replacement from the numbers corresponding to each class' rows. Then, a subset was created considering only the selected rows (last names).
Therefore, the new labeled dataset which is used through the classification methods consists of 12,496 last names separated as:

- Hispanic: 6,248 (50.0%).
- Non-Hispanic: 6,248 (50.0%).

3.2 *Data pre-processing*

As discussed in the methodology section, most ML classification methods take numerical values as the input data. Hence, we need to transform this categorical data into a numeric one. The method used for this transformation is the one-hot encoding. An X matrix of zeros was created having the same number of rows as the data set. A total number of 33 columns were created for each character of the last name (consisting of all possible letters

cols.→	*1*	*2*	*3*	*(+30)*	*34*	*35*	*36*	*(+30)*	*(×14)*	*529*	*530*	...	*561*
rows↓	A	B	C	...	A	B	C	...	...	A	B	...	...
1	0	0	0	...	0	0	0	...	...	0	0	...	0
...	...	...	...	...	...	...	...	...	...	...	...	...	...
12,596	0	0	0	...	0	0	0	...	...	0	0	...	0

Table 3: Representation of X matrix of zeros (dim: $12,596 \times 561$).

from the alphabet plus letters with accent marks). In this dataset, the longest last name is 17 characters long. Therefore, this X matrix has 12,596 rows $\times$ 561 columns. Table 3 shows a representation of this X matrix, identifying each column with the corresponding letter for each character with the longest last name size.

Next, the R function `CharToCode` was used inside a `for loop` to construct the corresponding binary matrix from the last names dataset. The algorithm works as follows. It goes through each character of each last name to match this character with the corresponding 1-out-of-33 possible letters. This matching is now written as a "1", showing the presence of the letter in the corresponding character position. So, for each continued 33 columns we only have a '1' and all the others columns are '0'. Finally, this sequence of 0's and 1's is replaced in the X matrix previously generated. The above steps are repeated for each last name of the dataset to be used in the classification methods.

Next, a Y vector of same rows dimension was created. This vector represents the two classes, being '0' the Hispanic class and '1' the non-Hispanic class, as shown in expression below. The X matrix and Y vector are bound together to get the entire dataset after applying the one hot encoding.

$$Y = \begin{cases} 0 & \text{, if "Hispanic"} \\ 1 & \text{, if "non-Hispanic"} \end{cases}$$

3.3 *Classification methods*

The dataset previously obtained, after applying one-hot encoding, was then split into a training and testing set. This was done with a random sample without replacement using the following proportions: 2/3 for the training and 1/3 for the testing set. Hence, we have an X training set and a Y training set as well as the testing sets, respectively. These

consists of 8,397 observations for the training, and 4,199 for the testing.

Next, classification methods were used using these training and testing sets for both model training and then predicting, respectively. The parameters and sample coding in each case are shown next:

- Logistic Regression: we used the `glm` function taking the Y training as the response and all X matrix columns as the predictors. The regression type is "binomial" as we are dealing with two classes. A sample of the coding is shown next: `glm(Ytrain~., family=binomial, data=XYmerge[Train,])`. Note that the data input is a merged dataframe of X and Y, but only including the train rows. Using this fitted model, we obtained predictions from the XY test observations, using the `predict` function as follows: `predict(lrmodel, XYmerge[Test,], type='response')`. The type 'response' is to output the probabilities in the form $P(Y = 1|X)$, this is the probability of an observation to belong to the "positive" class (James et al. (2021)).

- Naive Bayes: we used the function `naiveBayes` from `e1071` library, the input was the response vector Y, the XY dataset and the training subset. A sample of the coding is as follows: `naiveBayes(Y ~., data=XYmerge, subset=Train)`. From this fitted model, we predicted the classes from the XY test observations, using the function `predict(nbmodel, XYmerge[Test,])`.

- k-NN: we used the `knn` function from the `class` library to train and evaluate different models using a range of values of k, and computed the TPR and FPR for each model. Next, an ROC curve was plotted to identify the best parameter value selecting the smallest euclidean distance to the point (0,1), which corresponds to the largest possible TPR and the smallest FPR. Figure 2 shows the ROC curve for all computed values of k (1-267), highlighted in red the observation for best parameter k. This $k = 7$ was the hyperparameter used in the function, along with the X train and test sets and the Y train vector. The final model for the k-NN method was coded as follows: `knn(Xtrain, Xtest, Ytrain , k=7)`. The results from this fitted model gave the prediction for each Xtest observation as one of the two classes, "0" and "1".

- GLPL: the library `grpreg` has a function for the grouped penalty regression methods. We used the function `cv.grpreg` to train a classifier using the GLPL method

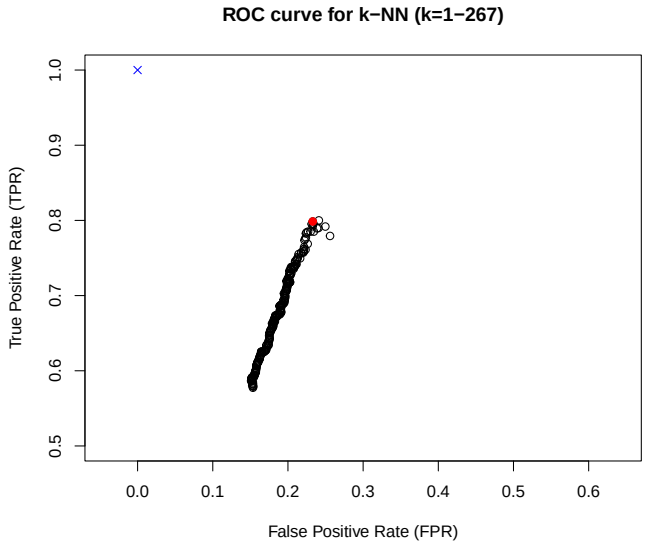

Figure 2: ROC curve for k-NN (varying k from 1 to 267). The red point is the observation corresponding to best parameter $k = 7$. This is based on the shortest euclidean distance to point (0,1) shown as a blue $\times$, which is the highest possible True Positive Rate (TPR) and lowest possible False Positive Rate (FPR).

and performing cross-validation (CV) to get the best penalty parameter. The CV was performed using 10-folds. We also had to specify which regressors we are interested in grouping, for this case we grouped by character position (from c1 to c17, the longest last name), so the last names sequence of characters (regressors) are kept all together. A sample code for this: `cv.grpreg(Xtrain,Ytrain,group,penalty="grLasso", family="binomial")`. From the model results we obtained the best penalization parameter with the `model$lambda.min` function, which resulted to be $\lambda = 0.00098$. Finally, we obtained the predictions from the test data using the function as shown next: `predict(cvglpl-model, Xtest, type="class")`

3.4 *Confusion matrix*

For each of the classification methods, a confusion matrix was computed using the `table` function. This confusion matrix is constructed based on the true class vs the predicted class on the testing set. A sample for this function is as follows: `table(Ypred,Ytest)`.

From here we get the four components of the confusion matrix. Our "overall" goal is to get a model with the lesser misclassification possible. Using these values, we calculated the following measures to compare the performance between all the classification methods used, as detailed on the methodology section 2.3.1: Accuracy, TPR, FPR.

3.5 *Classification on new-unbalanced data set*

Up to this point we already trained some models using the training set and then evaluated their performance on the testing set. We undersampled the majority class in order to obtained a balanced training data set. However, in real life applications we also want to perform well on a new dataset which most likely would be unbalanced.

Therefore, we tested each ML classifier on a new-unbalanced dataset, making predictions with the previously trained models and/or using the optimal hyperparameters for k-NN and GLPL methods.

The new-unbalanced dataset used is the remaining part of the original complete dataset, as we had 97,093 last names but only used 12,496 on the previous sections. Hence, the new-unbalanced dataset is composed by 84,597 last names labeled as:

- Hispanic: 6,249 (7.4%)
- Non-Hispanic: 78,348 (92.6%)

After obtaining the predictions for each classification method, a confusion matrix was computed and then the performance measures that have been utilized in this study (Accuracy, TPR, FPR) were calculated and compared.

4 Results & Discussion

The performance of the classifiers was evaluated using the confusion matrix results for each method. The overall performance is seen in terms of the correctly classified observations, either Hispanic or non-Hispanic; this is the model accuracy. However, we are also interested in comparing the proportion of correct and incorrect predictions for the *Hispanic* class specifically, these are the TPR and FPR measures.

Throughout these results, the 2 classes are presented as follows:

- Hispanic = true class "H", predicted class "$H*$".
- Non-Hispanic = true class "NH", predicted class "$NH*$".

4.1 *Classification using testing set*

4.1.1 *Logistic Regression*

Logistic regression classifier seems to overall perform well, as we can see from results of the confusion matrix in Table 4 mostly classifies correctly in both classes. It could also be noted that the Hispanic class is mainly correctly classified than not.

		True class		
		H	**NH**	
Predicted class	**H**	$1,748$	424	$H* = 2,172$
	NH	332	$1,662$	$NH* = 1,994$
		$H = 2,080$	$NH = 2,086$	Total $= 4,166$

Table 4: Confusion Matrix results for logistic regression classifier in testing set.

From this confusion matrix we can now compute the performance measures:

- Accuracy: $(1,748 + 1,662)/4,166 = 0.819$
- TPR: $1,748/2,080 = 0.840$
- FPR: $424/2,086 = 0.203$

4.1.2 *Naive Bayes*

The Naive Bayes classifier shows mostly correct predictions for the H class. However, the NH classification have a bigger error. Thus, the overall performance of the classifier is not good. Also, there could be seen that it predicts a significant number of H class that actually belong to the NH class. Results of the confusion matrix are shown in Table 5.

		True class		
		H	**NH**	
Predicted class	**H**	1,919	1,540	$H* = 3,459$
	NH	161	546	$NH* = 707$
		$H = 2,080$	$NH = 2,086$	Total $= 4,166$

Table 5: Confusion Matrix results for naive bayes classifier in testing set.

From this confusion matrix we can now compute the performance measures:

- Accuracy: $(1,919 + 546)/4,166 = 0.592$
- TPR: $1,919/2,080 = 0.923$
- FPR: $1,540/2,086 = 0.738$

4.1.3 *k-NN*

The k-NN classifier performs well, when using the best parameter k obtained from the ROC curve ($k = 7$). There could be seen that for both classes H and NH most of the observations are correctly classified. Results of the confusion matrix are shown in Table 6.

		True class		
		H	**NH**	
Predicted class	**H**	1,661	486	$H* = 2,147$
	NH	419	1,600	$NH* = 2,019$
		$H = 2,080$	$NH = 2,086$	Total $= 4,166$

Table 6: Confusion Matrix results for k-NN classifier in testing set.

From this confusion matrix we can now compute the performance measures:

- Accuracy: $(1,661 + 1,600)/4,166 = 0.783$
- TPR: $1,661/2,080 = 0.799$
- FPR: $486/2,086 = 0.233$

4.1.4 *GLPL*

The GLPL classifier shows a small misclassification error in both classes, when using the penalty parameter obtained by cross-validation ($\lambda = 0.00098$). Results of the confusion matrix are shown in Table 7.

		True class		
		H	**NH**	
Predicted class	**H**	$1,759$	426	$H* = 2,185$
	NH	321	$1,660$	$NH* = 1,981$
		$H = 2,080$	$NH = 2,086$	Total $= 4,166$

Table 7: Confusion Matrix results for GLPL classifier in testing set.

From this confusion matrix we can now compute the performance measures:

- Accuracy: $(1,759 + 1,660)/4,166 = 0.821$
- TPR: $1,759/2,080 = 0.846$
- FPR: $426/2,086 = 0.204$

4.2 *Classification on new unbalanced dataset*

4.2.1 *Logistic Regression*

The trained logistic regression classifier shows a small misclassification error in both classes, when making predictions in the unbalanced dataset. We could see that results are consistent when compared with the balanced testing set. Table 8 presents the confusion matrix results.

		True class		
		H	**NH**	
Predicted class	**H**	$5,177$	$16,373$	$H* = 21,550$
	NH	$1,072$	$61,975$	$NH* = 63,047$
		$H = 6,249$	$NH = 78,348$	Total $= 84,597$

Table 8: Confusion Matrix results for logistic regression classifier on new unbalanced dataset.

From this confusion matrix we can now compute the performance measures:

- Accuracy: $(5,177 + 61,975)/84,597 = 0.819$
- TPR: $5,177/6,249 = 0.828$
- FPR: $16,373/78,348 = 0.209$

4.2.2 *Naive Bayes*

The naive Bayes classifier have a poor performance due to the misclassification error, mainly on the NH class. Even though the H prediction is generally good, this significantly impacts the overall accuracy given the weight/proportion of the NH class in the unbalanced dataset. Confusion matrix results for this classifier are shown in table 9.

		True class		
		H	**NH**	
Predicted class	**H**	$5,780$	$58,141$	$H* = 63,921$
	NH	469	$20,207$	$NH* = 20,676$
		$H = 6,249$	$NH = 78,348$	Total $= 84,597$

Table 9: Confusion Matrix results for naive Bayes classifier on new unbalanced dataset.

From this confusion matrix we can now compute the performance measures:

- Accuracy: $(5,780 + 20,207)/84,597 = 0.307$
- TPR: $5,780/6,249 = 0.925$
- FPR: $58,141/78,348 = 0.742$

4.2.3 *k-NN*

The dataset must be splitted into a training and testing set due to how the algorithm works, given it is a non-parametric method as explained in section 2.2.3. The k-NN function returns a prediction to the testing set, learning from the training set input. Therefore, using the 2/3-1/3 same approach to split, the testing set consists of 28,199 observations. This is the size for the total results in the confusion matrix shown in table 10. Note that this k-NN

classifier is not performing well when making predictions on the unbalanced dataset, given that it mostly assigns observations into the majority class NH.

		True class		
		H	**NH**	
Predicted class	**H**	201	62	$H* = 263$
	NH	$1,871$	$26,065$	$NH* = 27,936$
		$H = 2,072$	$NH = 26,127$	Total $= 28,199$

Table 10: Confusion Matrix results for k-NN classifier on new unbalanced dataset.

From this confusion matrix we can now compute the performance measures:

- Accuracy: $(201 + 26,065)/28,199 = 0.931$
- TPR: $201/2,072 = 0.097$
- FPR: $62/26,127 = 0.002$

4.2.4 *GLPL*

The GLPL classifier seems to perform well overall and in each class, respectively. Results of the confusion matrix are presented in Table 11, where it could be seen that it is mostly correctly predicted in both classes.

		True class		
		H	**NH**	
Predicted class	**H**	$5,191$	$16,418$	$H* = 21,609$
	NH	$1,058$	$61,930$	$NH* = 62,988$
		$H = 6,249$	$NH = 78,348$	Total $= 84,597$

Table 11: Confusion Matrix results for GLPL classifier on new unbalanced dataset.

From this confusion matrix we can now compute the performance measures:

- Accuracy: $(5,191 + 61,930)/84,597 = 0.793$
- TPR: $5,191/6,249 = 0.831$
- FPR: $16,418/78,348 = 0.210$

4.3 *Comparison between methods*

Now that we have the performance measures - accuracy, TPR, FPR - for all the classifiers we compared them so we can select the method that performs better for this classification problem. Table 12 summarizes these three performance measures to compare between all classification methods when using the trained model on the testing set.

	Logistic Reg.	**Naive Bayes**	**k-NN** ($k = 7$)	**GLPL** ($\lambda = 0.00098$)
Accuracy	81.9%	59.2%	78.3%	82.1%
TPR	84.0%	92.3%	79.9%	84.6%
FPR	20.3%	73.8%	23.3%	20.4%

Table 12: Performance comparison between methods (testing set).

The performance measures obtained when making predictions on the new unbalanced dataset are presented in Table 13 to compare between all classification methods.

	Logistic Reg.	**Naive Bayes**	**k-NN** ($k = 7$)	**GLPL** ($\lambda = 0.00098$)
Accuracy	81.9%	30.7%	93.1%	79.3%
TPR	82.8%	92.5%	9.7%	83.1%
FPR	20.9%	74.2%	0.2%	21.0%

Table 13: Performance comparison between methods (new unbalanced dataset).

If we compare table 12 and table 13 we can see that logistic regression and GLPL classifiers have a very similar performance, being these the best classifiers. Also, there is consistency in the results from the testing set and the unbalanced dataset. However, we can not state the same for the remaining two classifiers. The naive Bayes classifier did not perform well in terms of Accuracy and FPR in none of the two datasets, therefore we may not rely in this method for a classification of this nature. On the other hand, the k-NN classifier performed well on the first case, which was the testing set from the balanced dataset. We can see that these three performance measures are very close to the logistic regression and GLPL, having slightly lower values. However, this classifier's performance was very low when using the new unbalanced dataset in terms of TPR and FPR, even when

using the same best hyperparameter $k = 7$. Note that Accuracy is not affected, it even increased, but this is only because the majority class (non-Hispanic) weight is much bigger than the minority class (Hispanic). Hence, even if Hispanic class misclassification rate is large, it did not significantly affect the overall accuracy of the classifier.

5 Conclusion and Further Work

The performance of several supervised ML classifiers was evaluated to predict ethnicity from last names. Input data was transformed from string to numerical data using one-hot encoding, and then these classification methods were used: logistic regression, naive Bayes, k-NN and grouped Lasso penalized logistic (GLPL). Based on our results, we can say that the logistic regression and GLPL classifiers are the best in terms of accuracy, true positive rate (TPR) and false positive rate (FPR). The k-NN classifier had a good performance only when having a balanced dataset. On the unbalanced dataset, TPR and FPR were significantly negatively affected. Therefore, whenever a classification is to be done using the k-NN method, getting a balanced dataset is crucial (either by undersampling on the majority class, or oversampling on the minority class). The naive Bayes classifier does not seem to be a reliable method for prediction of ethnicity from last names, considering the low performance measures obtained from the used dataset.

Potential applications of the classifier developed in this paper are:

1. Detecting bias in citations networks. Prediction of ethnicity using this classifier would allow to expand analysis of citation networks to identify ethnic groups as a potential source of bias.

2. Determining incidence of a disease among different ethnic groups. The classifier could be used for imputation of ethnicity in health care algorithms when not having this input from the patient.

3. Gender classification from first names. Note that the training data would be different from the one used in this study, however the methodology/algorithm could be used practically identically, with either a logistic regression or a GLPL classifier. A challenge, like in any ML classification problem, is to find reliable labeled databases to train and test the models.

A final area for further research is to investigate the use of an artificial neural network (NN) classifier, again, using the one-hot encoding used in this work. Given the success of "deep" neural networks (DNN) in processing text data, it seems reasonable to expect their performance, as the number of layers increases, should improve on GLPL, our best ML algorithm in this work. Other considerations, like the amount of training data a DNN would require to perform at such level and the training time involved, would have to be considered in practice.

References

Abu-Mostafa, Y. S. ., M. Magdon-Ismail, and H.-T. Lin (2012). *Learning from Data: A short course*. AMLBook.com.

Bagui, S., D. Nandi, S. Bagui, and R. J. White (2021, July). Machine learning and deep learning for phishing email classification using one-hot encoding. *Journal of Computer Science, 17(7), 610-623*.

Bertolero, M. A. e. a. (2020, October). Racial and ethnic imbalance in neuroscience reference lists and intersections with gender. *bioRxiv*.

Bishop, C. M. (2006). *Pattern Recognition and Machine Learning*. Springer.

Breheny, P. (2023, May). "Penalties". grpreg 3.4.0.3. URL: https://pbreheny.github.io/grpreg/articles/web/penalties.html.

Cabreros, I., D. Agniel, S. C. Martino, C. L. Damberg, and M. N. Elliott (2022). Predicting race and ethnicity to ensure equitable algorithms for health care decision making. *Health Affairs 41*(8), 1153–1159. PMID: 35914194.

DanB. (2017). "Using Categorical Data with One Hot Encoding". Kaggle website. URL: https://www.kaggle.com/code/dansbecker/using-categorical-data-with-one-hot-encoding.

Hu, Y., C. Hu, T. Tran, T. Kasturi, E. Joseph, and M. Gillingham (2021, May). What's in a name? – gender classification of names with character based machine learning models. *Data Mining and Knowledge Discovery 35, 1537–1563*.

James, G., D. Witten, T. Hastie, and R. Tibshirani (2021). *An Introduction to Statistical Learning with applications in R* (2nd ed.). Springer.

Kadir, M., P. Akash, S. Sharmin, A. Ali, and M. Shoyaib (2020, May). A proximity weighted evidential k nearest neighbor classifier for imbalanced data. In *Lauw, H., Wong, RW., Ntoulas, A., Lim, EP., Ng, SK., Pan, S. (eds) Advances in Knowledge Discovery and Data Mining. PAKDD 2020. Lecture Notes in Computer Science()*, Volume 12085. Springer, Cham.

Kangralkar, S. (2021, January). "Regularization and Cross-Validation — How to choose the penalty value (lambda)". Medium Analytics Vidhya. URL: https://medium.com/analytics-vidhya/regularization-and-cross-validation-how-to-choose-the-penalty-value-lambda-1217fa4351e5.

Lindholm, A., N. Wahlström, F. Lindsten, and T. B. Schön (2022). *Machine Learning A First Course for Engineers and Scientists.* Cambridge University Press.

Miller School of Medicine, U. o. M. (2023). "Data Acquisition Manual 2023. Appendix E: 2020 Census List of Spanish Surnames". The Florida Cancer Data System. URL: https://fcds.med.miami.edu/downloads/DataAcquisitionManual/dam2023/26

Newton, A. (2011). "last-names". ARIN Engineering GitHub public repository. URL: https://github.com/arineng/arincli/blob/master/lib/last-names.txt.

Shah, K., H. Patel, D. Sanghvi, and M. Shah (2020, March). A comparative analysis of logistic regression, random forest and knn models for the text classification. *Augmented Human Research 5.*

Shmueli, G., P. C. Bruce, I. Yahav, N. R. Patel, and K. C. Lichtendahl Jr. (2018). *Data Mining for Business Analytics. Concepts, Techniques, and Applications in R.* Wiley.

Singh, V. K., N. S. Maurya, A. Mani, and R. S. Yadav (2020). Machine learning method using position-specific mutation based classification outperforms one hot coding for disease severity prediction in haemophilia 'a'. *Genomics 112, Issue 6*, 5122–5128.

Xie, F. (2022, January). rethnicity: An r package for predicting ethnicity from names. *Software X Volume 17*, 100965.

Yuan, M. and Y. Lin (2006). Model selection and estimation in regression with grouped variables. *Journal of the Royal Statistical Society. Series B (Statistical Methodology).*

A Appendix: R script

This script implements the data pre-processing, and all training and testing of the different ML algorithms tried in the paper.

```
#install packages, load libraries
install.packages("dplyr")
install.packages("class")
install.packages("e1071")
install.packages("grpreg")
library(dplyr) #one hot encoding
library(class) #k-NN
library(e1071) #NaiveBayes
library(grpreg) #GLPL

# import dataframe
```

```
lastname_df <- read.csv("C:\\Users\\avc6470\\OneDrive - The Pennsylvania
                        State University\\PennState\\Research Paper\\
                        class_last_names.csv")
summary(lastname_df)

# checking missing values and removing
summary(is.na(lastname_df))
lastname_df <- na.omit(lastname_df)  #removing NAs
lastname_df$last_name <- toupper(lastname_df$last_name)  #uppercase
summary(is.na(lastname_df))

# indicator variables code - One hot encoding
namesY <- lastname_df
noRows <- dim(namesY)[1]
nHispanic <- sum(namesY[,2]=="hispanic")
nNonHispanic <- noRows-nHispanic
## Select a subset as  data
sizeTrainTest <- round(nHispanic/2)*2  #total sample size Hisp + nonHisp
maxLength <- max(nchar(namesY$last_name))
indices <- seq(1,noRows)
set.seed(3)
subsetH <- sample(indices[1:12497], size=round(sizeTrainTest/2))
subsetNH <-sample(indices[12498:97093], size=round(sizeTrainTest/2))
p <- nHispanic/noRows #proportion of Hisp in the sample
p
# Sample H and non H names randomly and put them together
S1 <- namesY[subsetH,]
S2 <- namesY[subsetNH,]
S <- rbind(S1,S2)
# Reshuffle final dataframe (S). This contains train+test data
set.seed(3)
S <- S[sample(sizeTrainTest),]
Y <- vector(length=sizeTrainTest)
for(i in 1:sizeTrainTest){
  if (S[i,2]=="hispanic"){
    Y[i] <- 0;
  }else{Y[i]<- 1}
} ##coding Hispanic="0", nonHisp="1"

# create a 1-of-K vector for each character in name and concatenate all
X <- matrix(nrow=sizeTrainTest,ncol=33*maxLength,0)
library(dplyr)
for (i in 1:sizeTrainTest){
  #Define a 33xmaxLength matrix for each name,
  ##we will convert it to a row of matrix X
  xrow <- matrix(nrow=33,ncol=maxLength,0)
```

```
  len <- nchar(S[i,1])
  for (j in 1:min(len,maxLength)){
    charToCode <- substr(toupper(S[i,1]),j,j)
    index <- case_match(
      charToCode,
      "A" ~1,
      "B" ~2,
      "C" ~3,
      "D" ~4,
      "E" ~5,
      "F" ~6,
      "G" ~7,
      "H" ~8,
      "I" ~9,
      "J" ~10,
      "K" ~11,
      "L" ~12,
      "M" ~13,
      "N" ~14,
      "Ñ" ~15,
      "O" ~16,
      "P" ~17,
      "Q" ~18,
      "R" ~19,
      "S" ~20,
      "T" ~21,
      "U" ~22,
      "V" ~23,
      "W" ~24,
      "Y" ~25,
      "X" ~26,
      "Z" ~27,
      "Á" ~28,
      "É" ~29,
      "Í" ~30,
      "Ó" ~31,
      "Ú" ~32,
      " " ~33
    )
    xrow[index,j] <- 1
  } #endfor j
  xrow <- c(xrow)
  X[i,]<- t(xrow)
} #endfor i ---> ~3min running time

# split X and Y into train/test
```

```
set.seed(3)
TrainRows <- sample.int(sizeTrainTest,sizeTrainTest*2/3,replace = FALSE)
TestRows <- setdiff(seq(1:sizeTrainTest),TrainRows)
Xtrain <- X[TrainRows,]
Ytrain <- Y[TrainRows]
Xtest <- X[TestRows,]
Ytest <- Y[TestRows]

# k-NN for a range of "k" values --> split if necessary (running time)
library(class)
n = 300
knn_error2 = rep(NA,n)
tpr2 = rep(NA,n)
fpr2 = rep(NA,n)
# do 1-100,101-200,201-300
for (m in 1:n){
  set.seed(2)
  knn_pred2 <- knn(Xtrain, Xtest, Ytrain , k=m)
  knn_error2[m] <- mean(knn_pred2 != Ytest)
  t2 <- table(knn_pred2, Ytest)
  tpr2[m] <- t2[1]/(t2[1]+t2[2])
  fpr2[m] <- t2[3]/(t2[3]+t2[4])
}
# tabulating misclassification error, tpr, fpr for all values of k
knn_error_df2 <- data.frame(k = 1:n2 , test_error = knn_error2,
                            TPR = tpr2, FPR = fpr2)
summary(knn_error_df2)  #NA's, error in model: too many ties in knn
knn_error_df2 <- na.omit(knn_error_df2)  ##removing NA's
summary(knn_error_df2)
# calculate euclidean distance from point (0,1) to each point
dist_knn <- sqrt(((knn_error_df2$TPR-1)^2)+((knn_error_df2$FPR)^2))
knn_error_df2 <- data.frame(knn_error_df2, dist_knn)
best_k <- which.min(knn_error_df2$dist_knn)  #minimum dist_knn
# ROC curve - using different values of k
plot(knn_error_df2$FPR, knn_error_df2$TPR, xlab="False Positive Rate (FPR)",
     ylab="True Positive Rate (TPR)", type="b",
     main="ROC curve for k-NN (k=1-267)")
points(knn_error_df2$FPR[best_k], knn_error_df2$TPR[best_k],
       col="red", pch=19)
# adjusting y-axis to 1, and same scale for x&y
plot(knn_error_df2$FPR, knn_error_df2$TPR, xlab="False Positive Rate (FPR)",
     ylab="True Positive Rate (TPR)", type="b",
     main="ROC curve for k-NN (k=1-267)",
     xlim=c(0.0,0.6), ylim=c(0.5,1.0), asp = 1)
points(knn_error_df2$FPR[best_k], knn_error_df2$TPR[best_k],
       col="red", pch=19)
```

```
points(0,1,col="blue",pch=4)

# predicting with best value of k (~35sec running time)
best_k   # best=7
knn_pred2 <- knn(Xtrain, Xtest, Ytrain, k=best_k)
## confusion matrix using k=best_k=7--> TPR=TP/P FPR=FP/N
t2 <- table(knn_pred2, Ytest)
t2
mean(Ytest == knn_pred2) #Accuracy
t2[1]/(t2[1]+t2[2])      #TPR
t2[3]/(t2[3]+t2[4])      #FPR

# NAIVE BAYES
library(e1071)
XYmerge <- as.data.frame(cbind(Y,X))
nb_fit <- naiveBayes(Y~., data= XYmerge , subset = TrainRows)
nb_pred <- predict(nb_fit , XYmerge[TestRows,])
t3 <- table(nb_pred, Ytest)
t3
mean(Ytest == nb_pred) #Accuracy
t3[1]/(t3[1]+t3[2])    #TPR
t3[3]/(t3[3]+t3[4])    #FPR

# LOGISTIC REGRESSION
lr_fit <- glm(Y~. , family=binomial, data=XYmerge[TrainRows,])
lr_pred <- round(predict(lr_fit , XYmerge[TestRows,], type="response"))
t4 <- table(lr_pred, Ytest)
t4
mean(Ytest == lr_pred) #Accuracy
t4[1]/(t4[1]+t4[2])    #TPR
t4[3]/(t4[3]+t4[4])    #FPR

# GLPL model  ~6min in total
library(grpreg)
# Define groups
##groups of indicator variables defining each character in the sequence,
###c1 to cmaxLen=17
group <- c(rep("c1",33),rep("c2",33),rep("c3",33),rep("c4",33),
           rep("c5",33),rep("c6",33),rep("c7",33),rep("c8",33),
           rep("c9",33),rep("c10",33),rep("c11",33),rep("c12",33),
           rep("c13",33),rep("c14",33),rep("c15",33),rep("c16",33),
           rep("c17",33))
glpl_fit <- grpreg(Xtrain,Ytrain,group,penalty="grLasso",family="binomial")
plot(glpl_fit)
# Do a 10-fold cross-validation fit to get best lambda
cvglpl_fit <- cv.grpreg(Xtrain, Ytrain, group, penalty="grLasso",
```

```
                        family="binomial")
cvglpl_fit$lambda.min #best penalization parameter
plot(cvglpl_fit)
# Now predict at test data and get confusion matrix
glpl_pred <-  predict(cvglpl_fit, Xtest, type="class")
t5 <- table(glpl_pred, Ytest)
t5
mean(Ytest == glpl_pred) #Accuracy
t5[1]/(t5[1]+t5[2])      #TPR
t5[3]/(t5[3]+t5[4])      #FPR

# Now classifying in the remaining part of the dataset

used_rows <- c(subsetH, subsetNH)
namesY_new <- namesY[-used_rows,] #new df excluding used last names
size <- nrow(namesY_new)
nHispanic_new <- sum(namesY_new[,2]=="hispanic")
nNonHispanic_new <- size-nHispanic_new
p_new <- nHispanic_new/size  #proportion of Hispanic in new dataset
p_new

S_new <- namesY_new[sample(size),]  #reshuffle df
Y_new <- vector(length=size)
for(i in 1:size){
  if (S_new[i,2]=="hispanic"){
    Y_new[i] <- 0;
  }else{Y_new[i]<- 1}
} ##coding Hispanic="0", nonHisp="1"

#One hot encoding ---> ~11min running time for 85k
X_new <- matrix(nrow=size,ncol=33*maxLength,0)
library(dplyr)
for (i in 1:size){
  #Define a 33xmaxLength matrix for each name,
  ##we will convert it to a row of matrix X
  xrow <- matrix(nrow=33,ncol=maxLength,0)
  len <- nchar(S_new[i,1])
  for (j in 1:min(len,maxLength)){
    charToCode <- substr(toupper(S_new[i,1]),j,j)
    index <- case_match(
      charToCode,
      "A" ~1,
      "B" ~2,
      "C" ~3,
      "D" ~4,
      "E" ~5,
```

```
      "F" ~6,
      "G" ~7,
      "H" ~8,
      "I" ~9,
      "J" ~10,
      "K" ~11,
      "L" ~12,
      "M" ~13,
      "N" ~14,
      "Ñ" ~15,
      "O" ~16,
      "P" ~17,
      "Q" ~18,
      "R" ~19,
      "S" ~20,
      "T" ~21,
      "U" ~22,
      "V" ~23,
      "W" ~24,
      "Y" ~25,
      "X" ~26,
      "Z" ~27,
      "Á" ~28,
      "É" ~29,
      "Í" ~30,
      "Ó" ~31,
      "Ú" ~32,
      " " ~33
    )
    xrow[index,j] <- 1
  } #endfor j
  xrow <- c(xrow)
  X_new[i,]<- t(xrow)
} #endfor i

# Predicting using trained models and new data

#Naive Bayes (nb_fit) ~7min
library(e1071)
XYmerge_new <- as.data.frame(cbind(Y_new,X_new))
nb_pred_new <- predict(nb_fit , XYmerge_new)
t8 <- table(nb_pred_new, Y_new)
t8
mean(Y_new == nb_pred_new) #Accuracy
t8[1]/(t8[1]+t8[2])        #TPR
t8[3]/(t8[3]+t8[4])        #FPR
```

```
#Logistic Reg (lr_fit) ~15sec
lr_pred_new <- round(predict(lr_fit, XYmerge_new, type="response"))
t9 <- table(lr_pred_new, Y_new)
t9
mean(Ytest == lr_pred) #Accuracy
t9[1]/(t9[1]+t9[2])    #TPR
t9[3]/(t9[3]+t9[4])    #FPR

#GLPL CV (cvglpl_fit) ~2sec
library(grpreg)
glpl_pred_new <- predict(cvglpl_fit, X_new, type="class")
t10 <- table(glpl_pred_new, Y_new)
t10
mean(Y_new == glpl_pred_new) #Accuracy
t10[1]/(t10[1]+t10[2])       #TPR
t10[3]/(t10[3]+t10[4])       #FPR

#kNN (k=7) we need train and test sets ~45min
library(class)
set.seed(3)
train <- sample(1:size, round(size*2/3))
test <- -train
knn_pred_new <- knn(X_new[train,], X_new[test,], Y_new[train], k=7)
t11 <- table(knn_pred_new, Y_new[test])
t11
mean(Y_new[test] == knn_pred_new) #Accuracy
t11[1]/(t11[1]+t11[2])            #TPR
t11[3]/(t11[3]+t11[4])            #FPR
```